RETAILMART V3 · SQL

Cohort, RFM & Funnel Analysis

— Siraj Sir

📱 WhatsApp Announcement

COHORT ANALYSIS, RFM & FUNNELS

Every growth team in the world asks three questions. (1) Of the customers who signed up in January, how many are still active 6 months later? (Cohort retention.) (2) Who are our most valuable customers? (RFM segmentation.) (3) Of the users who landed on the homepage, how many actually checked out? (Funnel analysis.) Today you answer all three.

Today's Focus:

- Cohort retention matrix — signup month × months-since
- RFM segmentation — Champions, Loyal, At-Risk, Lost
- Funnel queries — drop-off at each step
- CLV — customer lifetime value per acquisition channel

This is the analyst layer that drives growth-team decisions at every major Indian D2C company.

— Siraj Sir

#Day27 #PostgreSQL #Cohort #RFM #Funnel

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	Data quality + dedup recap, How clean data feeds today's analytics
Part 1: Cohort Retention	40 mins	Signup cohorts — what they are, why they matter, Months-since-signup grid, Pivoting cohort buckets into a matrix, 2 practice problems
Part 2: RFM Segmentation	40 mins	Recency, Frequency, Monetary — the three pillars, NTILE-based scoring (1-5 per dimension), Customer segments: Champions / Loyal / At-Risk / Lost, 2 practice problems
Part 3: Funnels & CLV	35 mins	Step-by-step user funnel, Drop-off rates and conversion %, Customer Lifetime Value (CLV) per channel, 2 practice problems
Quiz + Summary	20 mins	Interview-style questions, Homework, Interview Prep preview

YOUR SQL JOURNEY

- ✓ Pivoting + FILTER
- ✓ Median + DISTINCT ON
- ✓ Date/Time Mastery
- ✓ JSON / JSONB
- ✓ Views & MVs
- ✓ Data Quality
- 27 **Cohort, RFM, Funnels** ← HERE
- 28 Review + Interview Prep

RECAP

Memory Check

Finding duplicates with GROUP BY HAVING and ROW_NUMBER. Deduping while keeping the right record. NULL strategies, regex pattern matching, and anomaly flags.

Today: With clean, deduped data, you can finally do the analyst's most valuable work — cohort retention, customer segmentation, and funnel drop-off analysis. This is what growth teams pay analysts for.

STORY

The Growth Director's Whiteboard

Every Monday morning, the Growth Director at any D2C company writes the same three questions on the whiteboard:

'Of the customers we acquired in January, what percent are still buying in July?' — that's cohort retention.

'Who are our 100 most valuable customers right now — who deserves a hand-written thank-you card?' — that's RFM segmentation.

'Of the 10,000 people who landed on the product page yesterday, how many actually paid? Where did the other 9,000 drop off?' — that's funnel analysis.

Three patterns. One analyst can compute all of them by lunchtime — if they know SQL.

REVEAL

From Whiteboard to SQL!

'Still active after N months' → cohort grid (signup month × months-since)

'Top X% customers' → RFM with NTILE(5) on Recency, Frequency, Monetary

'Where did users drop off?' → funnel CTEs with cumulative counts

'CLV by channel' → SUM(net_total) per acquisition source

All four patterns combine CTEs + window functions + JOINs

STORY

The College Reunion Attendance

Imagine tracking college reunion attendance. Class of 2020: 200 graduates. How many came to year-1 reunion? Year-5? Year-10? Now do the same for Class of 2021, 2022, 2023. Stack each year's class as a ROW, and 'years since graduation' as COLUMNS. That's a COHORT GRID. Same logic for customer retention.

DEFINITION

Cohort Retention

Technical:
A cohort is a group of users sharing a common acquisition period (usually month). Retention is measured per cohort: 'of users who signed up in cohort X, how many were active N months later?' Result is typically pivoted into a matrix: rows = cohorts, columns = months-since-signup, cells = retention %.

Layman's:
Cohort = a graduating class. Retention = how many came back for each reunion. The grid lets you see if newer cohorts retain BETTER or WORSE than older ones — the most important question in growth analytics.

SYNTAX: Cohort Grid (Skeleton) (1/2)

```
WITH cohorts AS (
  SELECT cust_id AS customer_id,
    DATE_TRUNC('month',
      MIN(order_date)) AS cohort_month
  FROM sales.orders
  GROUP BY cust_id
),
activity AS (
  SELECT c.customer_id,
    c.cohort_month,
    DATE_TRUNC('month', o.order_date)
      AS active_month,
    EXTRACT(MONTH FROM AGE(
      o.order_date, c.cohort_month))::INT
      AS months_since
  FROM cohorts c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
)
SELECT cohort_month, months_since,
```

SYNTAX: Cohort Grid (Skeleton) (2/2)

```
  COUNT(DISTINCT customer_id) AS active
FROM activity
GROUP BY cohort_month, months_since
ORDER BY cohort_month, months_since;
```

Easy: Signup-Cohort Retention Grid (Months 0–6)

EASY

SCENARIO: The Growth Director wants the canonical cohort grid: for each signup month, what % of those customers placed an order N months later? Cover months 0 through 6. This is the single most-asked cohort query at any D2C company.

YOUR TASK: CTE 1: customers + their cohort_month (first order's month). CTE 2: join orders, compute months_since per row. CTE 3: aggregate distinct customers per (cohort, months_since). Pivot using FILTER for months 0-6.

🤔 **Hint:** Cohort retention = three CTEs in a row — definition, activity, pivot. Trap: every customer is automatically active in month 0 of their cohort (it's literally their first order); ALWAYS verify month-0 retention is ~100% as a sanity check on your join logic.

Answer (1/3)

✅ ANSWER

```
WITH cohorts AS (
  SELECT cust_id AS customer_id,
    DATE_TRUNC('month',
      MIN(order_date))::DATE AS cohort_month
  FROM sales.orders
  GROUP BY cust_id
),
activity AS (
  SELECT c.customer_id,
    c.cohort_month,
    EXTRACT(YEAR FROM AGE(
      o.order_date, c.cohort_month)) * 12
    + EXTRACT(MONTH FROM AGE(
      o.order_date, c.cohort_month))
      AS months_since
  FROM cohorts c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
),
cohort_sizes AS (
  SELECT cohort_month,
    COUNT(DISTINCT customer_id)
```

Answer (2/3)

✅ ANSWER

```
      AS cohort_size
  FROM cohorts
  GROUP BY cohort_month
)
SELECT
  a.cohort_month,
  cs.cohort_size,
  ROUND(100.0 * COUNT(DISTINCT
    CASE WHEN months_since = 0
      THEN a.customer_id END)
    / cs.cohort_size, 1) AS m0,
  ROUND(100.0 * COUNT(DISTINCT
    CASE WHEN months_since = 1
      THEN a.customer_id END)
    / cs.cohort_size, 1) AS m1,
  ROUND(100.0 * COUNT(DISTINCT
    CASE WHEN months_since = 3
      THEN a.customer_id END)
    / cs.cohort_size, 1) AS m3,
  ROUND(100.0 * COUNT(DISTINCT
    CASE WHEN months_since = 6
      THEN a.customer_id END)
```

Answer (3/3)

✅ ANSWER

```
    / cs.cohort_size, 1) AS m6
FROM activity a
JOIN cohort_sizes cs
  ON a.cohort_month = cs.cohort_month
GROUP BY a.cohort_month, cs.cohort_size
ORDER BY a.cohort_month LIMIT 12;
```

Hard: Product-Category Cross-Purchase Matrix

HARD

SCENARIO: The CMO wants to know: customers who bought Apparel, what % ALSO bought Footwear? This 'cross-purchase' matrix powers cross-sell recommendations and bundling strategy.

YOUR TASK: Build CTEs grouping customers by category bought. SELF JOIN the category-customer mapping. Count customers in BOTH categories (Cat A AND Cat B). Express as % of Cat A customers.

🤔 **Hint:** Cross-purchase = SELF JOIN on a per-customer category list. Trap: a customer who bought BOTH categories must NOT count themselves — use an inequality (cat1 != cat2) to avoid the diagonal in the matrix; otherwise every category shows 100% 'cross-purchase' with itself.

Answer (1/2)

✅ ANSWER

```
WITH cust_cats AS (
  SELECT DISTINCT
    o.cust_id,
    cat.category_name
  FROM sales.orders o
  JOIN sales.order_items oi
    ON o.order_id = oi.order_id
  JOIN products.products p
    ON oi.prod_id = p.product_id
  JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
  JOIN core.dim_category cat
    ON b.category_id = cat.category_id
),
category_sizes AS (
  SELECT category_name,
    COUNT(DISTINCT cust_id) AS cat_size
  FROM cust_cats
  GROUP BY category_name
)
SELECT
  c1.category_name AS bought_a,
```

Answer (2/2)

✅ ANSWER

```
  c2.category_name AS also_bought_b,
  COUNT(DISTINCT c1.cust_id)
    AS shared_customers,
  ROUND(100.0 * COUNT(DISTINCT c1.cust_id)
    / cs.cat_size, 1) AS pct_of_a
FROM cust_cats c1
JOIN cust_cats c2
  ON c1.cust_id = c2.cust_id
  AND c1.category_name != c2.category_name
JOIN category_sizes cs
  ON cs.category_name = c1.category_name
GROUP BY c1.category_name,
  c2.category_name, cs.cat_size
ORDER BY pct_of_a DESC LIMIT 20;
```

STORY

The VIP Wedding Card List

A wedding planner has 500 contacts. The bride and groom can only invite 80. The planner ranks each contact on three things: how RECENT did we last talk? How FREQUENT do we meet? How meaningful is the bond? The top 80 across all three become invitees. Customer analytics works the same way.

DEFINITION

RFM — Recency, Frequency, Monetary

Technical:

RFM scores each customer on three dimensions: Recency (days since last purchase), Frequency (number of orders), Monetary (total spend). Each dimension is bucketed into 1-5 using NTILE. Champions = (5,5,5). Lost = (1,1,1). Used for targeted marketing, retention campaigns, and CLV prioritization.

Layman's:

RFM = three numbers per customer. High recency = they bought recently. High frequency = they buy often. High monetary = they spend a lot. Combining the three gives a customer's 'value score' for prioritized outreach.

SYNTAX: RFM with NTILE(5) (1/2)

```
WITH rfm_raw AS (
  SELECT cust_id,
    CURRENT_DATE - MAX(order_date)
      AS days_since_last,
    COUNT(*) AS frequency,
    SUM(net_total) AS monetary
  FROM sales.orders
  GROUP BY cust_id
),
rfm_scored AS (
  SELECT cust_id,
    NTILE(5) OVER (
      ORDER BY days_since_last DESC) AS r,
    NTILE(5) OVER (
      ORDER BY frequency ASC) AS f,
    NTILE(5) OVER (
      ORDER BY monetary ASC) AS m
  FROM rfm_raw
)
SELECT cust_id, r, f, m,
```

SYNTAX: RFM with NTILE(5) (2/2)

```
  r * 100 + f * 10 + m AS rfm_score
FROM rfm_scored;
```

Medium: RFM Scoring with Champion/Loyal/At-Risk/Lost Labels

MEDIUM

SCENARIO: The CMO wants every customer tagged with one of four buckets: Champions (top across all three), Loyal (high R+F, mid M), At-Risk (low R, but historically high F+M), Lost (low R+F+M). Used to launch retention campaigns.

YOUR TASK: Three CTEs: raw RFM metrics, NTILE-scored RFM (1-5 each), and labeled segment. Use CASE on the (R,F,M) scores to bucket. Show customer name, R, F, M, total spend, and segment.

🤔 **Hint:** RFM segmentation needs careful NTILE direction — recency is REVERSED (most recent = highest score), frequency and monetary are NORMAL (more = higher score). Trap: getting the NTILE direction wrong on recency turns Champions into Lost and vice versa; double-check the ORDER BY direction.

Answer (1/2)

✅ ANSWER

```
WITH rfm_raw AS (
  SELECT cust_id,
    CURRENT_DATE - MAX(order_date)::DATE
      AS days_since,
    COUNT(*) AS frequency,
    SUM(net_total) AS monetary
  FROM sales.orders
  GROUP BY cust_id
),
rfm_scored AS (
  SELECT cust_id, days_since,
    frequency, monetary,
    NTILE(5) OVER (
      ORDER BY days_since DESC) AS r,
    NTILE(5) OVER (
      ORDER BY frequency ASC) AS f,
    NTILE(5) OVER (
      ORDER BY monetary ASC) AS m
  FROM rfm_raw
)
SELECT
  c.first_name || ' ' || c.last_name
```

Answer (2/2)

✅ ANSWER

```
    AS customer,
  rs.days_since,
  rs.frequency, rs.r, rs.f, rs.m,
  ROUND(rs.monetary, 0) AS lifetime_spend,
  CASE
    WHEN r >= 4 AND f >= 4 AND m >= 4
      THEN 'Champion'
    WHEN r >= 3 AND f >= 3
      THEN 'Loyal'
    WHEN r <= 2 AND f >= 4
      THEN 'At-Risk'
    WHEN r <= 2 AND f <= 2
      THEN 'Lost'
    ELSE 'Average'
  END AS segment
FROM rfm_scored rs
JOIN customers.customers c
  ON rs.cust_id = c.customer_id
ORDER BY rs.r DESC, rs.f DESC, rs.m DESC
LIMIT 20;
```

Crazy: CLV per Acquisition Source (Loyalty Tier)

CRAZY

SCENARIO: The CMO wants Customer Lifetime Value broken down by loyalty tier (a proxy for acquisition channel). Per tier: avg CLV, top quintile CLV, # of customers, total revenue contribution. Helps decide where to spend acquisition budget.

YOUR TASK: CTE 1: per-customer CLV (lifetime spend). CTE 2: join to loyalty.members + loyalty.tiers. Aggregate per tier with AVG, PERCENTILE_CONT(0.8), COUNT, SUM. Sort by avg_clv DESC.

🤔 **Hint:** CLV per group is just SUM(net_total) per customer, then AVG/percentile per tier. Trap: customers who aren't loyalty members fall out with INNER JOIN; if the question is 'all customers by channel', LEFT JOIN with COALESCE('Non-Member') so they show up in their own bucket.

Answer (1/2)

✅ ANSWER

```
WITH cust_clv AS (
  SELECT cust_id,
    SUM(net_total) AS lifetime_value,
    COUNT(*) AS lifetime_orders
  FROM sales.orders
  GROUP BY cust_id
)
SELECT
  COALESCE(t.tier_name, 'Non-Member')
    AS tier,
  COUNT(*) AS customers,
  ROUND(AVG(cc.lifetime_value), 0)
    AS avg_clv,
  ROUND((PERCENTILE_CONT(0.8)
    WITHIN GROUP (ORDER BY
    cc.lifetime_value))::NUMERIC, 0)
    AS p80_clv,
  ROUND(SUM(cc.lifetime_value), 0)
    AS tier_total_revenue,
  ROUND(AVG(cc.lifetime_orders), 1)
    AS avg_orders
FROM cust_clv cc
```

Answer (2/2)

✅ ANSWER

```
LEFT JOIN loyalty.members m
  ON cc.cust_id = m.customer_id
LEFT JOIN loyalty.tiers t
  ON m.tier_id = t.tier_id
GROUP BY t.tier_name
ORDER BY avg_clv DESC;
```

STORY

The Cricket Stadium Gates

100,000 fans buy tickets. 95,000 actually show up. 90,000 pass through Gate 1. 80,000 pass through security. 70,000 reach their seat. Each step DROPS some fans. The biggest drop tells you where to invest — fix the slowest gate first. Funnels are exactly this in customer data.

DEFINITION

Funnel Analysis

Technical:
Funnel analysis counts distinct users at each sequential step (e.g., signed up → viewed product → added to cart → checked out → paid). Drop-off rate between steps reveals where users abandon. Built as CTE chains where each step's user set is a subset of the previous.

Layman's:
Funnel = a sequence of yes/no checks. Each step asks: 'of users who did X, how many ALSO did Y?' The biggest drop = the biggest improvement opportunity for the product team.

SYNTAX: Funnel CTE Pattern (1/2)

```
WITH step1_signed_up AS (
  SELECT customer_id
  FROM customers.customers
  WHERE registration_date >= '2024-01-01'
),
step2_placed_order AS (
  SELECT DISTINCT cust_id
  FROM sales.orders
  WHERE cust_id IN (
    SELECT customer_id FROM step1_signed_up)
),
step3_two_plus_orders AS (
  SELECT cust_id
  FROM sales.orders
  WHERE cust_id IN (
    SELECT cust_id FROM step2_placed_order)
  GROUP BY cust_id HAVING COUNT(*) >= 2
)
SELECT 'Step 1: Signed Up' AS step,
  COUNT(*) FROM step1_signed_up
```

SYNTAX: Funnel CTE Pattern (2/2)

```
UNION ALL ...;
```

Medium: Signup → First Order → Second Order Funnel

MEDIUM

SCENARIO: The Product Manager wants the classic activation funnel: how many customers signed up, then how many placed their first order, then how many came back for a SECOND. Conversion % at each step.

YOUR TASK: Three CTEs (one per step). UNION ALL the step counts. Add a column showing % retained from previous step and from the top.

🤔 **Hint:** Each funnel step is a SUBSET of the previous step — order matters and each CTE depends on the prior. Trap: don't compute conversion % using the original signup count for every step; the 'step-to-step' rate uses the IMMEDIATELY PREVIOUS step as the denominator, not the top.

Answer (1/2)

✅ ANSWER

```
WITH signed_up AS (
  SELECT customer_id
  FROM customers.customers
),
first_order AS (
  SELECT DISTINCT cust_id
    AS customer_id
  FROM sales.orders
),
two_plus_orders AS (
  SELECT cust_id AS customer_id
  FROM sales.orders
  GROUP BY cust_id
  HAVING COUNT(*) >= 2
),
step_counts AS (
  SELECT 1 AS step_num,
    'Signed Up' AS step_name,
    COUNT(*) AS users
  FROM signed_up
  UNION ALL
  SELECT 2, 'Placed First Order',
```

Answer (2/2)

✅ ANSWER

```
    COUNT(*) FROM first_order
  UNION ALL
  SELECT 3, 'Placed Second Order',
    COUNT(*) FROM two_plus_orders
)
SELECT step_num, step_name, users,
  ROUND(100.0 * users
    / FIRST_VALUE(users)
      OVER (ORDER BY step_num), 1)
    AS pct_of_top,
  ROUND(100.0 * users
    / LAG(users) OVER (
      ORDER BY step_num), 1)
    AS pct_of_prev
FROM step_counts
ORDER BY step_num;
```

Hard: DAU/MAU Stickiness Metric from Web Events

HARD

SCENARIO: Product wants the 'stickiness' metric: DAU (Daily Active Users) ÷ MAU (Monthly Active Users). A stickiness of 0.5 means the average user comes back 15 days per month. This is the SaaS engagement gold standard, computed from web_events.page_views.

YOUR TASK: CTE 1: distinct (customer_id, view_date) pairs from page_views. CTE 2: AVG daily distinct customers (DAU). CTE 3: distinct customers in the month (MAU). Compute DAU/MAU ratio.

🤔 **Hint:** DAU is an AVERAGE daily distinct count across the period — not the latest day. MAU is the distinct count for the WHOLE month. Trap: anonymous visitors (customer_id IS NULL) should be excluded — a session without a known user can't count as the same user across days.

Answer (1/2)

✅ ANSWER

```
WITH daily AS (
  SELECT
    DATE_TRUNC('day',
      view_timestamp)::DATE AS day,
    COUNT(DISTINCT customer_id)
      AS daily_users
  FROM web_events.page_views
  WHERE customer_id IS NOT NULL
    AND view_timestamp
      >= CURRENT_DATE - INTERVAL '30 days'
  GROUP BY DATE_TRUNC('day',
    view_timestamp)
),
month_total AS (
  SELECT
    COUNT(DISTINCT customer_id) AS mau
  FROM web_events.page_views
  WHERE customer_id IS NOT NULL
    AND view_timestamp
      >= CURRENT_DATE - INTERVAL '30 days'
)
SELECT
```

Answer (2/2)

✅ ANSWER

```
  ROUND(AVG(d.daily_users)) AS dau,
  mt.mau,
  ROUND(AVG(d.daily_users)
    / NULLIF(mt.mau, 0), 3)
    AS stickiness
FROM daily d
CROSS JOIN month_total mt
GROUP BY mt.mau;
```

INTERVIEW

What is a cohort retention matrix?

Q: 'Explain a cohort retention matrix and how you would build one.'

A: A 2D grid where rows = signup cohorts (typically by month) and columns = months-since-signup. Each cell shows what % of that cohort was still active N months later. Built with 3 CTEs: define cohorts (first activity per user), measure activity per (user, month), pivot to grid. Used to spot retention regressions in newer cohorts.

INTERVIEW

What's the difference between RFM and CLV?

Q: 'How do RFM and CLV differ?'

A: RFM is a CURRENT-state scoring across three dimensions — Recency, Frequency, Monetary — used for segmentation right now. CLV (Customer Lifetime Value) is a TOTAL spend or PROJECTED future spend per customer. RFM helps prioritize who to target THIS WEEK. CLV helps prioritize acquisition channels and tier-based investment.

INTERVIEW

Walk through a funnel query

Q: 'Write SQL for a 3-step user funnel.'

A: Chain CTEs — each step filters down from the previous: WITH step1 AS (...), step2 AS (SELECT ... WHERE id IN step1), step3 AS (SELECT ... WHERE id IN step2). UNION ALL the per-step counts. Add conversion % using LAG or FIRST_VALUE. The biggest drop-off step is your highest-leverage improvement target.

INTERVIEW

Why NTILE for RFM?

Q: 'Why use NTILE(5) for RFM rather than fixed thresholds?'

A: NTILE distributes customers EQUALLY into buckets — always gives you 20% in each. Fixed thresholds (e.g., monetary > 50K = top tier) skew with data drift; what was 'top 20%' last year might be 'top 5%' this year. NTILE adapts automatically. Use fixed thresholds only when the business rule itself is absolute (e.g., 'VIP = ≥ 1L lifetime spend').

INTERVIEW

Real interview: Stickiness > 1?

Q: 'Can DAU/MAU ever exceed 1?'

A: No. DAU = distinct users on ONE day. MAU = distinct users across 30 days. Same user counted once in MAU regardless of how many days they came. AVG(DAU) over the period is BOUNDED above by MAU. If your calculation shows stickiness > 1, you have a bug — usually using SUM instead of AVG for daily counts, or not deduping users.

INTERVIEW

Cross-purchase matrix use case

Q: 'When would you build a cross-purchase matrix?'

A: Bundling, cross-sell campaigns, and category-affinity analysis. Example: 'customers who bought Footwear are 60% likely to buy Apparel within 30 days' → drives a cross-sell email. The matrix is just SELF JOIN on per-customer category list, divided by category size.

INTERVIEW

CLV and acquisition channel

Q: 'How would you decide which acquisition channel to invest more in?'

A: Compute CLV per acquisition source. Channel with HIGHEST CLV-per-customer is most valuable per acquisition. Cross-reference with CAC (Customer Acquisition Cost) — channel with highest CLV/CAC ratio is best ROI. Be careful with COHORT EFFECTS — newer customers haven't had time to accumulate spend; compare same-cohort age across channels.

KEY TAKEAWAYS

KEY TAKEAWAYS (1/2)

Cohort Retention:

1. Three CTEs: cohorts, activity, pivot to matrix.
2. Month 0 retention is always ~100% (sanity check).
3. Compare cohorts to spot retention regressions over time.

RFM Segmentation:

4. NTILE(5) on Recency (reversed!), Frequency, Monetary.
5. Champions = (5,5,5). Lost = (1,1,1). At-Risk = low R + high F+M.
6. NTILE adapts to data drift; fixed thresholds don't.

KEY TAKEAWAYS (2/2)

Funnels:

7. CTE chain where each step is a subset of the previous.
8. Two conversion rates: of-top and of-previous.
9. Biggest drop = biggest improvement opportunity.

CLV:

10. Per-customer SUM(net_total). Group by channel/tier for comparison.

REAL WORLD

What You Achieved Today

You learned the three pillars of growth analytics: cohort retention, RFM segmentation, funnel drop-off. Plus CLV calculations and the canonical stickiness metric.

These four patterns are what growth teams at Swiggy, Zomato, Flipkart, Razorpay, Cred, and every D2C company in India use DAILY. Master these and you walk into any analyst interview with the senior-analyst toolkit.

Tomorrow: Review + Interview Prep. We consolidate everything from earlier sessions into rapid-fire interview-style problems and the 'explain your query line by line' drill that interviews always test.

CHALLENGE

Cohort & RFM Take-Home Challenge (1/3)

Build these on RetailMart V3:

Easy:

1.: Monthly signup-cohort retention grid (months 0-6) for customers whose first order was in 2024.

2.: Cross-purchase matrix: top 5 category pairs by % of shared customers.

Medium:

CHALLENGE

Cohort & RFM Take-Home Challenge (2/3)

3.: RFM scoring with Champion/Loyal/At-Risk/Lost labels for all customers.

4.: Signup → first-order → second-order funnel with conversion %.

Hard:

5.: CLV per loyalty tier (including Non-Member). Show avg, p80, total.

6.:

CHALLENGE

Cohort & RFM Take-Home Challenge (3/3)

Stickiness (DAU/MAU) from web_events.page_views over the last 30 days.

Push your SQL file to GitHub!
https://github.com/starlordali4444/PostgreSql

STORY

Coming Tomorrow

Rapid-fire interview problems — top-N per group, MoM growth %, cohort retention, dedup with ROW_NUMBER, EXISTS vs IN, median per group, pivot with FILTER, recursive CTE, EXPLAIN plan reading.

Plus the 'Explain Your Query' drill: pick any query from the course and walk through it line by line, the way a senior interviewer expects. Interview Prep prepares you to walk into ANY analyst interview confident.